Things to look out for in solving the questions are:

- Make sure to name functions and arguments as stipulated in the question, but never be afraid to create extra functions of your own, e.g. to break up the code into conceptual sub-parts, or avoid redundancy in your code
- Commenting of code is one thing that you will be marked on; get some practice writing comments in your code, focusing on:
    - Describing key variables when they are first defined (but not things like index variables in `for` loops)
    - Describing what "chunks" of code do (i.e. not every line, but chunks of code that perform a particular operation, such as `# find the maximum value in the list` or `# count the number of vowels`.
    - Describing what every function does, including what its arguments are, and what it returns.

Note: If you make multiple submissions, only the most recent submission will be marked.

Scenario

Robotic Vacuum Cleaners are an increasingly popular tool for keeping homes and business clean. We wish to program a new robotic vacuum cleaner: The Vacuuminator.

The Vacuuminator

Representation

Like many small robotic devices, the Vacuuminator represents the world around it as a grid of small squares. In Python, this grid can be represented as a list of lists. Every position in the list of lists is represented by a string containing a single upper case character as follows:

- `'X'`: The location of the Vacuuminator.

- `'D'`: Contains dirt that the Vacuuminator should clean.
- `'W'`: A wall that the Vacuuminator cannot pass through.
- `'E'`: An empty square that the Vacuuminator can pass through but does not need to clean.

For example, the world below could be represented as

```
world = [['D', 'E', 'D', 'E'],
['E', 'W', 'W', 'W'],
['D', 'W', 'E', 'E'],
['E', 'D', 'E', 'X']]
```

Our representation

Note that `world[0][0]` therefore corresponds to the position in the top-left corner of the world. A 4 x 4 world is shown, but the Vacuuminator can handle worlds of different sizes. Note that the world will always be rectangular (i.e. there will never be rows of different length in the world).

The Vacuuminator can move up, down, left and right within the world. Each move can be represented by a single character:

- `'u'` for an upwards move, towards the 0th row of the world.
- `'d'` for a downwards move, away from the 0th row of the world.
- `'l'` for a left move, towards the 0th column of the world.
- `'r'` for a right move, away from the 0th column of the world.

If the Vacuuminator moves into a square containing dirt, it will clean the dirt, leaving the square empty once the Vacuuminator moves on.

Throughout this assignment you should assume there is only ever one Vacuuminator operating in the world.

Task 1: Distance to the nearest wall (3 marks)

The Vacuuminator needs to figure out how far it needs to travel to reach the nearest wall.

Write a function `distance_to_wall(world)`. This function should return an integer representing the smallest number of moves the Vacuuminator can make before hitting a wall.

If there are no walls present in the world, your function should return `None`.

Hint: Think about how you can calculate the number of moves the Vacuuminator can make before hitting a wall by using the number of rows and columns between the Vacuuminator and the wall. Also see the lecture recording on 25/3/2024 for some more guidance if you are having trouble.

Example Calls

```
>>> distance_to_wall([['D', 'W'], ['X', 'E']])
2

>>> distance_to_wall([['W', 'E'], ['X', 'E']])
1

>>> distance_to_wall([['D', 'D'], ['D', 'D'], ['W', 'W'], ['D', 'E'], ['W',
'E'], ['E', 'D'], ['E', 'X']])
3
```

Task 2: Making a Move (3 marks)

We now need to allow the Vacuuminator to move around the world. Write a function `make_move(world, direction)`. The `direction` may be one of `'u'`, `'d'`, `'l'`, or `'r'` as described on the Background slide. Your function return a list of lists representing the world after the move has been made.

Note that a move may not be possible either because:

- It would take the Vacuuminator off the edge of the world
- It would take the Vacuuminator into a wall

If an impossible move is requested, the Vacuuminator should not attempt to make it and should instead return the current state of the world.

Example Calls

```
>>> make_move([['E', 'W'], ['E', 'X']], 'l')
[['E', 'W'], ['X', 'E']]
>>> make_move([['E', 'W'], ['E', 'X']], 'r')
```

```
[['E', 'W'], ['E', 'X']]
>>> make_move([['E', 'W'], ['E', 'X']], 'u')
[['E', 'W'], ['E', 'X']]
```

Task 3: Finding Dirt (4 marks)

In the previous questions we assumed the Vacuuminator had a perfect knowledge of the world around it. In practice, mobile robots need to rely on sensors to find out information about the world. In this question we assume that the Vacuuminator has exactly four sensors:

- A 'left sensor' which will identify the nearest dirty square immediately to the left of the Vacuuminator (i.e. in the same row with a lower index position).
- A 'right sensor' which will identify the nearest dirty square immediately to the right of the Vacuuminator (i.e. in the same row with a higher index position).
- An 'up sensor' which will identify the nearest dirty square immediately above the Vacuuminator (i.e. in the same column with a lower index position).
- A 'down sensor' which will identify the nearest dirty square immediately below the Vacuuminator (i.e. in the same column with a higher index position).

The Vacuuminator's sensors are unable to penetrate walls, i.e. dirt on the opposite side of a wall will not be detected. The Vaccuminator is also unable to identify dirt that its sensors can't see (e.g. dirt that is diagonal to its current position).

Write a function `path_to_next(world)` that will identify the nearest piece of dirt that will be detected using the Vacuuminator's sensors. Your function should return a list containing the moves required for the Vacuuminator to reach that piece of dirt from its current position.

In the event that there are multiple pieces of dirt that are at an equal distance from the Vacuuminator, they should be prioritised in the following order:

- Highest Priority: Dirt above the Vacuuminator
- Second Priority: Dirt to the right of the Vacuuminator
- Third Priority: Dirt below the Vacuuminator
- Lowest Priority: Dirt to the left of the Vacuuminator

The Vaccumunator stays at its position when scanning, then goes straight to the nearest target dirt (which is directly at up, right, down or left directions from the vaccumator’s current position, and the target maybe one or more unit distances further away).

If no dirt can be detected, your function should return an empty list.

Example Calls

```
>>> print(path_to_next([['E', 'D'], ['E', 'X']]))
['u']

>>> print(path_to_next([['E', 'D'], ['E', 'E'], ['E', 'X']]))
['u', 'u']

>>> print(path_to_next([['D', 'E'], ['E', 'X']]))
[]
```

Task 4: Cleaning all the dirt located (5 marks)

We now wish to produce a sequence of moves to clean all of the dirt the Vacuuminator can find, and then return the Vacuuminator to its starting position. The Vacuuminator should repeatedly scan for and clean up dirt found from its current position according to the rules in Task 3. It should continue to do so as long as it finds new dirt from its current position. We will call this a cleaning cycle.

Once the Vacuuminator is no longer able to find any new dirt, it should backtrack along its path, scanning for new dirt at each square. If it finds any new dirt in its scan it should begin a new cleaning cycle, starting with this dirt.

The Vacuuminator should continue until it has backtracked through every move it has made and found no more dirt, scanning and cleaning as it goes. Note that once the Vacuuminator has selected a patch of dirt to clean, it will not scan for any new dirt until that patch has been cleaned. Scanning for new dirt will only happen while the Vacuuminator is backtracking.

Write a function `clean_path(world)`. This function should return a list of moves that the Vacuuminator will use to clean the world according to the rules above.

A working version of the Task 3 `path_to_next(world)` function is provided to help you with this task.

Example calls

```
>>> clean_path([['E', 'D'], ['E', 'E'], ['E', 'X']])
['u', 'u', 'd', 'd']

>>> clean_path([['D', 'D'], ['E', 'E'], ['E', 'X']])
['u', 'u', 'l', 'r', 'd', 'd']

>>> clean_path([['E', 'D', 'E'], ['X', 'D', 'D'], ['E', 'E', 'E']])
['r', 'u', 'd', 'r', 'l', 'l']

>>> clean_path([['E', 'D', 'E'], ['X', 'D', 'E'], ['E', 'D', 'E']])
['r', 'u', 'd', 'd', 'u', 'u', 'd', 'l']

>>> clean_path([['E', 'D', 'D'], ['D', 'E', 'E'], ['D', 'E', 'X']])
['u', 'u', 'l', 'r', 'd', 'l', 'l', 'd', 'u', 'r', 'r', 'd']
```